LegalEase ： AI Powered Legal Document Generator

Detailed Project Report

Executive Summary

LegalEase is an AI-powered legal document generation platform designed to address the critical accessibility gap in legal documentation for individuals, small businesses, and legal professionals across India. The system leverages advanced machine learning capabilities, retrieval-augmented generation (RAG), and multi-agent orchestration to automate the creation, review, and simplification of legal documents while maintaining human oversight and regulatory compliance.

The project addresses a fundamental challenge: legal documentation remains inaccessible to a vast majority of Indians due to complexity, cost, and time requirements. LegalEase proposes a solution that combines template-based generation, plain-language summarization, and seamless integration with existing legal resources to democratize access to legal document creation .

Core Value Proposition: Generate accurate, jurisdiction-aware legal documents in minutes rather than hours, with built-in verification mechanisms and human-in-the-loop controls that satisfy emerging regulatory requirements in India .

1. Introduction

1.1 Background

The Indian legal system processes millions of documents daily—contracts, affidavits, notices, petitions, and agreements. Yet the tools available for document creation remain largely manual, requiring either expensive professional expertise or risky self-drafting. The emergence of large language models (LLMs) presents an unprecedented opportunity to bridge this gap, provided that accuracy, accountability, and regulatory compliance are properly addressed.

1.2 Problem Statement

Three interconnected problems define the current landscape:

Accessibility: Individuals and small businesses lack affordable access to quality legal document drafting. Traditional legal services cost Rs. 5,000–50,000 for basic documents, placing them beyond reach for many .

Efficiency: Legal professionals spend disproportionate time on routine document drafting—work that is repetitive, template-driven, and amenable to automation. AI can generate standard documents like contracts and NDAs in minutes instead of hours .

Accuracy Risk: Unregulated AI use in legal contexts has produced documented harms. Indian courts have identified fabricated precedents, hallucinated citations, and synthetic legal authority as serious threats to judicial integrity . The Punjab & Haryana High Court recently quashed a GST show cause notice prepared primarily using AI without statutory sanction .

1.3 Proposed Solution

LegalEase addresses these problems through a controlled generation architecture that combines:

· Template-based document assembly with customizable parameters

· RAG (Retrieval-Augmented Generation) grounded in verified legal corpora

· Multi-agent workflow orchestration (Researcher, Drafter, Verifier, Reviewer)

· Plain-language summarization and risk flagging

· Mandatory human verification gates before document finalization

The system is designed from the ground up to comply with the Supreme Court of India's draft AI Regulations, 2026, the Bar Council of India's digital ethics circular, and the Digital Personal Data Protection Act, 2023 .

2. Market Analysis

2.1 Target Users

Primary Market: Small businesses and individuals in India requiring routine legal documents (rental agreements, NDAs, employment contracts, loan agreements, affidavits). The platform's focus on accessibility and user-friendliness targets users without formal legal training .

Secondary Market: Legal professionals seeking efficiency gains in document drafting and review. AI tools can identify errors, inconsistencies, and potential legal issues, reducing risk of human oversight

Tertiary Market: Law students and paralegals requiring document creation tools for educational and practical training purposes.

2.2 Market Size

India's legal services market is estimated at over $2 billion, with document drafting and review comprising a significant portion. The addressable market for AI-assisted legal document tools is projected to grow substantially as regulatory frameworks mature and acceptance increases.

2.3 Competitive Landscape

Existing solutions fall into three categories:

Category Examples Limitations

Template libraries LegalZoom, Rocket Lawyer Limited customization, no AI generation

Generic AI writing tools ChatGPT, Claude No legal grounding, hallucination risk, no templates

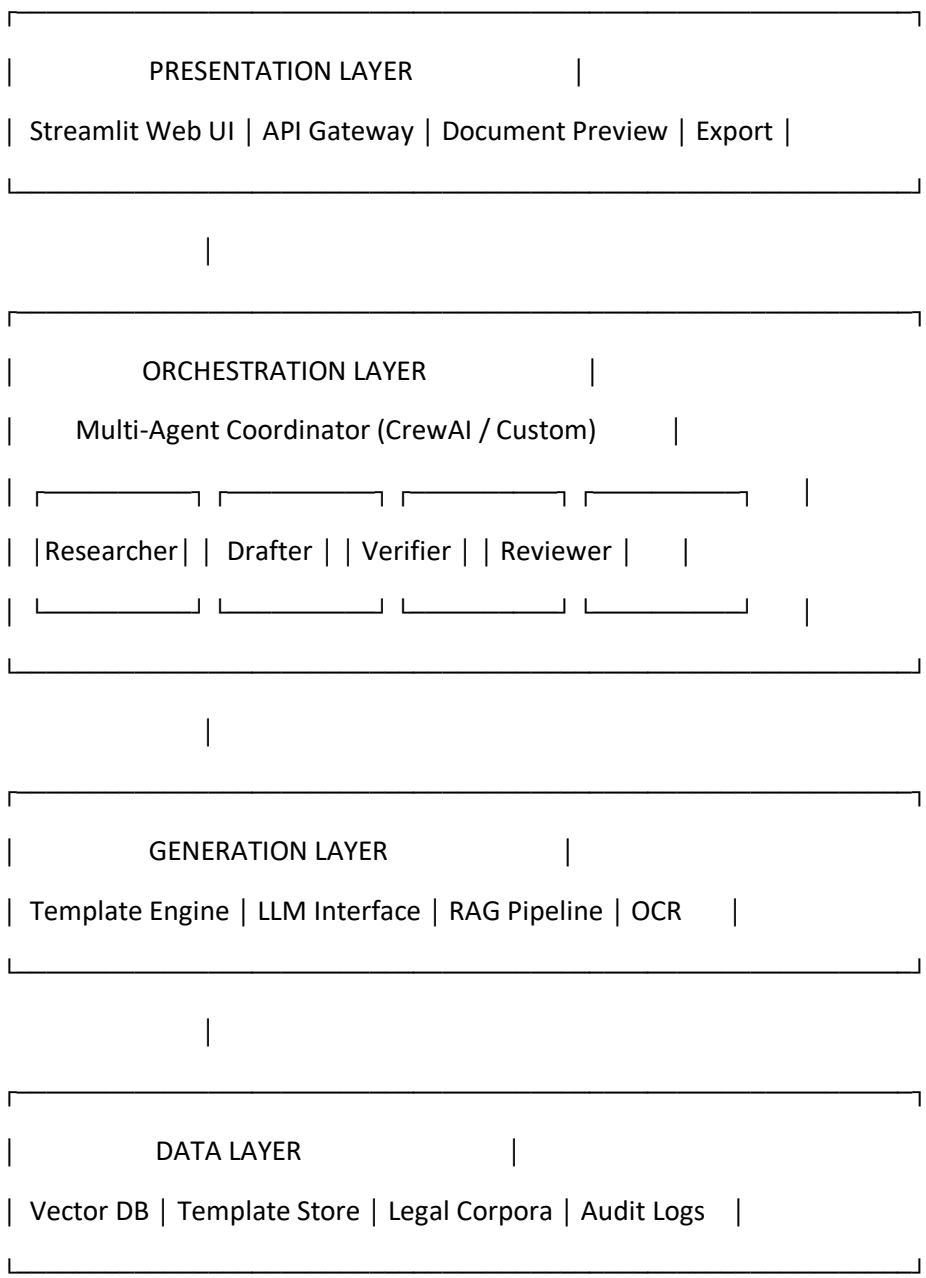
Specialized legal AI Harvey, Casetext Enterprise-focused, expensive, US-centric

LegalEase differentiates through India-specific legal grounding, accessibility-first design, and regulatory compliance by design.

3. Technical Architecture

3.1 System Overview

The LegalEase architecture follows a layered design that separates concerns between user interaction, orchestration, generation, and verification.



3.2 Core Components

Document Segmentation Layer: Automatically partitions legal documents into functional sections aligned with predefined templates using supervised classification techniques .

RAG (Retrieval-Augmented Generation) Layer: Expands textual suggestions by retrieving relevant sections from a corpus of legal documents and incorporating AI-generated content. This hybrid

approach ensures generated content remains contextually relevant and aligns with past legal decisions .

Multi-Agent System: Specialized agents collaborate to extract, reconcile, and generate high-quality content with minimal human oversight. Agents include Planner, Retriever, Summarizer, Synthesizer, and Reviewer .

Verification Engine: Cross-references generated citations against verified legal databases, flags potential hallucinations, and enforces mandatory human review gates.

3.3 Technology Stack

Layer Technology Purpose

UI Streamlit / React Web interface, document preview

Orchestration CrewAI / LangChain Multi-agent coordination

LLM GPT-4 / Llama 3 / DeepSeek Text generation

Vector Store ChromaDB / Pinecone Legal document retrieval

OCR Tesseract / PyMuPDF Document ingestion

Backend FastAPI / Python API services

Storage PostgreSQL / S3 Document and metadata persistence

---

4. Multi-Agent Workflow

4.1 Agent Roles

The system employs four specialized agents working in sequence with human checkpoints:

Agent 1: Legal Researcher

· Role: Retrieve relevant statutes, precedents, and template provisions

· Input: Document type, jurisdiction, key parameters

· Output: Structured legal context with verified citations

· Tools: RAG pipeline, legal database queries

Agent 2: Document Drafter

· Role: Generate document content based on template and research

· Input: Legal context, user parameters, template structure

· Output: Draft document with sections and provisions

· Tools: LLM, template engine, clause library

Agent 3: Citation Verifier

· Role: Validate all citations and legal references

· Input: Draft document with citations

· Output: Verification report with flagged issues

· Tools: Legal citation database, web verification (limited)

Agent 4: Plain-Language Reviewer

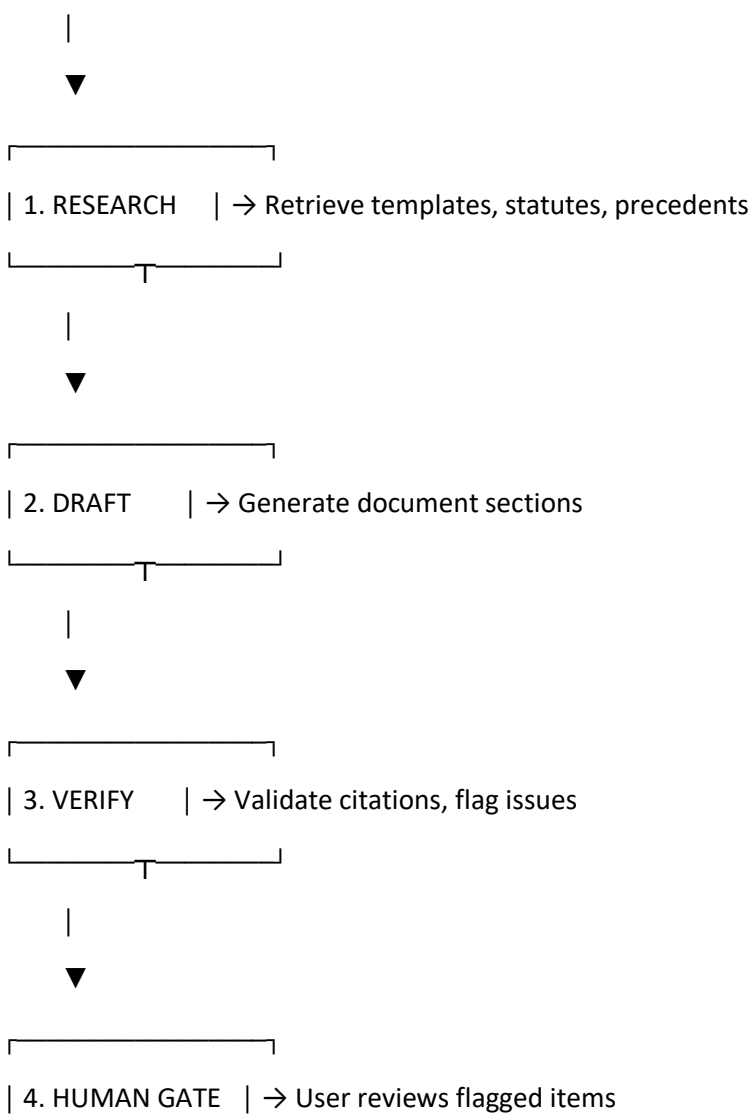
· Role: Generate simplified summary and risk flags

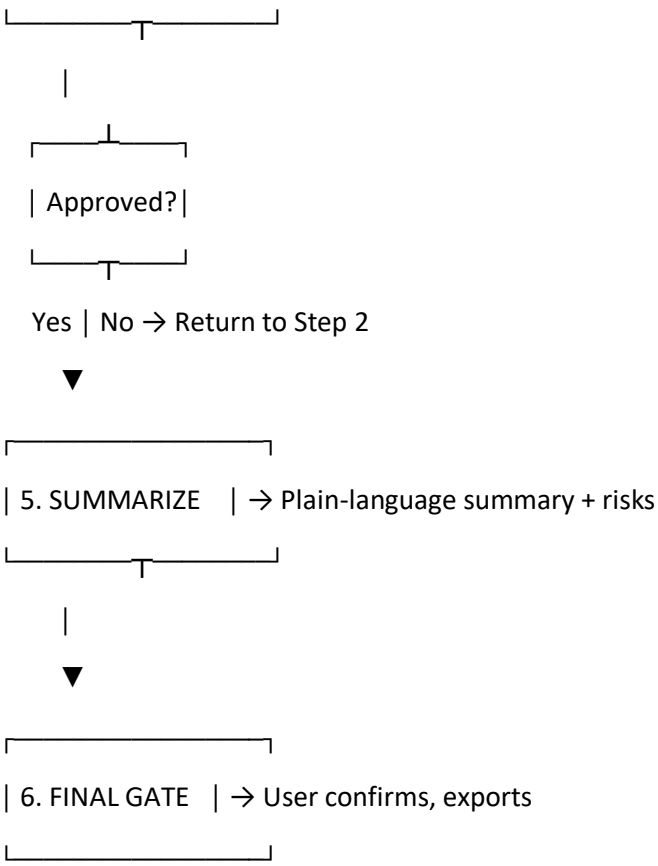
· Input: Verified draft document

· Output: Plain-language summary, risk assessment

· Tools: LLM summarization, risk rules engine

4.2 Workflow Sequence





4.3 Human-in-the-Loop Design

The Supreme Court of India has established that AI may assist, but responsibility must remain human . LegalEase implements this principle through:

· Mandatory review gates: No document is finalized without explicit human approval

· Verification disclosure: All AI-assisted content is marked and disclosed

· Audit trail: Every generation, modification, and approval is logged

· Citation confirmation: Users must confirm they have independently verified all legal citations

---

5. Implementation Code

5.1 Project Structure

```
legalease/
├── app.py                # Streamlit main application
├── agents/
│   ├── __init__.py
│   ├── researcher.py     # Legal research agent
│   ├── drafter.py        # Document drafting agent
```

```
|   ├── verifier.py          # Citation verification agent
|   └── reviewer.py          # Plain-language review agent
├── core/
|   ├── __init__.py
|   ├── rag.py               # RAG pipeline
|   ├── templates.py         # Template management
|   ├── verification.py      # Citation verification logic
|   └── ocr.py               # Document ingestion
├── templates/               # Document templates (YAML)
|   ├── nda.yaml
|   ├── rental_agreement.yaml
|   └── employment_contract.yaml
├── data/
|   ├── legal_corpus/        # Indexed legal documents
|   └── vector_store/        # ChromaDB persistence
├── config.yml               # Configuration
├── requirements.txt
└── .env                     # API keys
```

5.2 Core Agent Implementation

```python
# agents/researcher.py
from crewai import Agent, Task, LLM
from core.rag import LegalRAGPipeline
from typing import Dict, List
import yaml
class LegalResearcher:
    """Agent responsible for retrieving legal context and templates."""

    def __init__(self, config_path: str = "config.yml"):
        with open(config_path, 'r') as f:
            self.config = yaml.safe_load(f)
```

```
    self.rag = LegalRAGPipeline(
        corpus_path=self.config['corpus']['path'],
        vector_store_path=self.config['vector_store']['path']
    )

    self.agent = Agent(
        role="Legal Research Specialist",
        goal="Retrieve accurate, jurisdiction-appropriate legal provisions and templates",
        backstory="""You are an expert legal researcher with deep knowledge of
        Indian statutory law. You NEVER fabricate citations. If uncertain,
        you explicitly flag the need for human verification.""",
        verbose=True,
        allow_delegation=False
    )

def research(self, document_type: str, jurisdiction: str = "India",
             key_terms: List[str] = None) -> Dict:
    """
    Retrieve relevant legal context for document generation.

    Returns structured research output with:
    - Relevant template provisions
    - Applicable statutes and sections
    - Verified citations (or flags for unverified)
    """
    # Retrieve template provisions
    template_results = self.rag.retrieve(
        query=f"{document_type} template provisions",
        collection="templates",
        top_k=5
```

```
)

# Retrieve applicable statutes
statute_results = self.rag.retrieve(
    query=f"{document_type} applicable statutes {jurisdiction}",
    collection="statutes",
    top_k=5
)

# Retrieve relevant precedents (with verification flag)
precedent_results = self.rag.retrieve(
    query=f"{document_type} case law {jurisdiction}",
    collection="precedents",
    top_k=3
)

return {
    "template_provisions": template_results,
    "applicable_statutes": statute_results,
    "precedents": [
        {
            **p,
            "verification_required": True,
            "verification_note": "Must be independently verified before use"
        }
        for p in precedent_results
    ],
    "research_notes": self._generate_research_notes(
        document_type, template_results, statute_results
    )
}
```

```
    def _generate_research_notes(self, doc_type: str, templates, statutes) -> str:
        """Generate structured research notes for the drafter."""
        notes = f"Document Type: {doc_type}\n\n"
        notes += "Relevant Template Provisions:\n"
        for t in templates:
            notes += f"- {t.get('section', 'General')}: {t.get('content', '')[:100]}...\n"

        notes += "\nApplicable Statutes:\n"
        for s in statutes:
            notes += f"- {s.get('act', 'Unknown')}, Section {s.get('section', 'N/A')}\n"

        return notes
```

5.3 RAG Pipeline Implementation

```
python
# core/rag.py
import chromadb
from chromadb.config import Settings
from sentence_transformers import SentenceTransformer
from typing import List, Dict
import hashlib
class LegalRAGPipeline:
    """
    Retrieval-Augmented Generation pipeline for legal documents.

    Design principle: All retrieval is grounded in verified legal corpora.
    The system NEVER generates citations from model memory alone.
    """

    def __init__(self, corpus_path: str, vector_store_path: str):
        self.client = chromadb.PersistentClient(
```

```
        path=vector_store_path,
        settings=Settings(anonymized_telemetry=False)
    )
    self.encoder = SentenceTransformer('all-MiniLM-L6-v2')
    self.collections = {}
    self._initialize_collections()

def _initialize_collections(self):
    """Initialize or load existing collections."""
    for name in ["templates", "statutes", "precedents", "clauses"]:
        try:
            self.collections[name] = self.client.get_collection(name)
        except:
            self.collections[name] = self.client.create_collection(
                name=name,
                metadata={"hnsw:space": "cosine"}
            )

def index_document(self, collection: str, doc_id: str,
                   content: str, metadata: Dict):
    """Index a legal document into the vector store."""
    embedding = self.encoder.encode(content).tolist()

    # Generate content hash for integrity verification
    content_hash = hashlib.sha256(content.encode()).hexdigest()

    self.collections[collection].add(
        ids=[doc_id],
        embeddings=[embedding],
        documents=[content],
        metadatas=[{
```

```
            **metadata,
            "content_hash": content_hash,
            "verified": metadata.get("verified", False)
        }]
    )

def retrieve(self, query: str, collection: str,
             top_k: int = 5) -> List[Dict]:
    """
    Retrieve relevant documents for a query.

    Critical: Returns verification status for all retrieved items.
    Unverified items are clearly flagged for human review.
    """
    query_embedding = self.encoder.encode(query).tolist()

    results = self.collections[collection].query(
        query_embeddings=[query_embedding],
        n_results=top_k,
        include=["documents", "metadatas", "distances"]
    )

    retrieved = []
    for i, doc in enumerate(results['documents'][0]):
        metadata = results['metadatas'][0][i]
        retrieved.append({
            "content": doc,
            "metadata": metadata,
            "similarity": 1 - results['distances'][0][i],
            "verified": metadata.get("verified", False),
            "source": metadata.get("source", "Unknown")
```

```
            })

        return retrieved

    def verify_citation(self, citation: str) -> Dict:
        """
        Verify a legal citation against indexed corpus.

        Returns verification status. If the citation cannot be found
        in verified sources, it is flagged as UNVERIFIED and must
        not be relied upon without independent verification.
        """
        results = self.retrieve(citation, "precedents", top_k=3)

        if not results:
            return {
                "status": "NOT_FOUND",
                "message": "Citation not found in verified corpus. DO NOT RELY without independent verification.",
                "verified": False
            }

        best_match = results[0]

        if best_match['verified'] and best_match['similarity'] > 0.9:
            return {
                "status": "VERIFIED",
                "matched_content": best_match['content'][:200],
                "source": best_match['source'],
                "verified": True
            }
```

```
        else:
            return {
                "status": "UNCERTAIN",
                "message": f"Partial match found (similarity: {best_match['similarity']:.2f}). Manual verification required.",
                "candidate": best_match['content'][:200],
                "verified": False
            }
```

5.4 Document Drafter Agent

```
python
# agents/drafter.py
from crewai import Agent, Task
from core.templates import TemplateEngine
from typing import Dict, List
import yaml

class DocumentDrafter:
    """Agent responsible for generating document content."""

    def __init__(self, config_path: str = "config.yml"):
        with open(config_path, 'r') as f:
            self.config = yaml.safe_load(f)

        self.template_engine = TemplateEngine("templates/")

        self.agent = Agent(
            role="Legal Document Drafter",
            goal="Generate complete, accurate legal documents based on research and templates",
            backstory="""You are an expert legal drafter with extensive experience
            in Indian contract law and legal documentation. You follow templates
            precisely, incorporate researched provisions accurately, and NEVER
```

```
        invent legal language. When uncertain, you flag the issue for review.""",
        verbose=True
    )

def draft(self, document_type: str, parameters: Dict,
          research: Dict) -> Dict:
    """
    Generate a complete document draft.

    Process:
    1. Load appropriate template
    2. Populate template variables with parameters
    3. Enhance with researched provisions
    4. Mark all AI-generated content for disclosure
    """
    # Load template
    template = self.template_engine.load(document_type)

    # Prepare template variables
    variables = self._prepare_variables(parameters, research)

    # Render template sections
    sections = []
    for section_def in template['sections']:
        section_content = self.template_engine.render_section(
            section_def, variables
        )

        sections.append({
            "name": section_def['name'],
            "content": section_content,
```

```
            "ai_generated": section_def.get('ai_enhance', True),
            "requires_review": section_def.get('human_review', False)
        })

    # Generate document metadata
    metadata = {
        "document_type": document_type,
        "jurisdiction": parameters.get("jurisdiction", "India"),
        "generated_at": self._timestamp(),
        "ai_disclosure": "This document was generated with AI assistance. "
                         "All content must be independently verified before use.",
        "version": "1.0"
    }

    return {
        "metadata": metadata,
        "sections": sections,
        "parameters": parameters,
        "research_references": research.get('applicable_statutes', [])
    }

def _prepare_variables(self, parameters: Dict, research: Dict) -> Dict:
    """Prepare template variables from parameters and research."""
    variables = {
        "today": self._timestamp(),
        **parameters
    }

    # Add researched provisions as optional enhancement
    if research.get('template_provisions'):
        variables['researched_provisions'] = research['template_provisions']
```

```
        return variables

    def _timestamp(self) -> str:
        from datetime import datetime
        return datetime.now().strftime("%B %d, %Y")
```

5.5 Verification Agent

```
python
# agents/verifier.py
from crewai import Agent
from core.rag import LegalRAGPipeline
from typing import Dict, List
import re

class CitationVerifier:
    """
    Agent responsible for verifying all citations and legal references.

    Critical function: This agent implements the Supreme Court's mandate
    that no AI-generated legal material be relied upon without verification.
    """

    def __init__(self, config_path: str = "config.yml"):
        with open(config_path, 'r') as f:
            self.config = yaml.safe_load(f)

        self.rag = LegalRAGPipeline(
            corpus_path=self.config['corpus']['path'],
```

```
        vector_store_path=self.config['vector_store']['path']
    )

    self.agent = Agent(
        role="Legal Citation Verifier",
        goal="Ensure all legal citations are accurate, verifiable, and properly sourced",
        backstory="""You are a meticulous legal citation verifier. You know
        that fabricated precedents have caused serious harm in Indian courts.
        You NEVER approve a citation without verification. When uncertain,
        you flag the citation as UNVERIFIED and recommend human review.""",
        verbose=True
    )

def verify_document(self, document: Dict) -> Dict:
    """
    Verify all citations in a document.

    Returns:
    - List of verified citations
    - List of flagged citations requiring human verification
    - Overall verification status
    """
    all_citations = self._extract_citations(document)

    verified = []
    flagged = []

    for citation in all_citations:
        result = self.rag.verify_citation(citation['text'])

        if result['status'] == 'VERIFIED':
```

```
            verified.append({
                **citation,
                "verification": result
            })
        else:
            flagged.append({
                **citation,
                "verification": result,
                "action_required": "MANDATORY human verification before use"
            })

    # Determine overall status
    if flagged:
        overall_status = "REQUIRES_HUMAN_VERIFICATION"
    elif verified:
        overall_status = "ALL_VERIFIED"
    else:
        overall_status = "NO_CITATIONS"

    return {
        "overall_status": overall_status,
        "verified_count": len(verified),
        "flagged_count": len(flagged),
        "verified_citations": verified,
        "flagged_citations": flagged,
        "disclosure": self._generate_disclosure(verified, flagged)
    }

def _extract_citations(self, document: Dict) -> List[Dict]:
    """Extract all legal citations from document sections."""
    citations = []
```

```
    # Citation patterns for Indian legal documents
    patterns = [
        r'(?:Section|Sec\.)\s+\d+[A-Z]?(?:\s+of\s+the\s+[A-Za-z\s]+Act,?\s*\d{4})',
        r'\(\d{4}\)\s+\d+\s+SCC\s+\d+',
        r'\d{4}\s+SCC\s+OnLine\s+\w+\s+\d+',
        r'[A-Z][a-z]+\s+v\.\s+[A-Z][a-z]+',
        r'Article\s+\d+(?:\s+of\s+the\s+Constitution)?'
    ]

    for section in document.get('sections', []):
        content = section.get('content', '')

        for pattern in patterns:
            matches = re.finditer(pattern, content)
            for match in matches:
                citations.append({
                    "text": match.group(),
                    "section": section['name'],
                    "position": match.start()
                })

    return citations

def _generate_disclosure(self, verified: List, flagged: List) -> str:
    """Generate mandatory AI disclosure statement."""
    disclosure = "AI DISCLOSURE: This document contains AI-generated content. "

    if verified:
        disclosure += f"{len(verified)} citation(s) were verified against the legal corpus. "
```

```
    if flagged:
        disclosure += f"{len(flagged)} citation(s) could not be fully verified and "
        disclosure += "REQUIRE INDEPENDENT HUMAN VERIFICATION before use. "

    disclosure += ("The user assumes full responsibility for verifying all content. "
                   "AI-generated material must not be relied upon as legal authority "
                   "without independent verification.")

    return disclosure
```

5.6 Streamlit Application

python

```
# app.py
import streamlit as st
import yaml
from agents.researcher import LegalResearcher
from agents.drafter import DocumentDrafter
from agents.verifier import CitationVerifier
from agents.reviewer import PlainLanguageReviewer
import json

# Page configuration
st.set_page_config(
    page_title="LegalEase - AI Legal Document Generator",
    page_icon="⚖️",
    layout="wide"
)

# Load configuration
```

```
@st.cache_resource
def load_components():
    """Initialize agents (cached to avoid reloading)."""
    return {
        "researcher": LegalResearcher(),
        "drafter": DocumentDrafter(),
        "verifier": CitationVerifier(),
        "reviewer": PlainLanguageReviewer()
    }

def main():
    st.title("⚖️ LegalEase")
    st.subheader("AI-Powered Legal Document Generator")

    # Sidebar - Document Configuration
    with st.sidebar:
        st.header("Document Settings")

        doc_type = st.selectbox(
            "Document Type",
            ["Non-Disclosure Agreement", "Rental Agreement",
             "Employment Contract", "Loan Agreement", "Affidavit"]
        )

        jurisdiction = st.selectbox(
            "Jurisdiction",
            ["India - Central", "Maharashtra", "Delhi", "Karnataka",
             "Tamil Nadu", "Gujarat"]
        )

        st.divider()
```

```
        st.caption("⚠ All AI-generated content requires human verification")

    # Main content area
    col1, col2 = st.columns([1, 1])

    with col1:
        st.header("Document Parameters")

        # Dynamic parameter inputs based on document type
        if doc_type == "Non-Disclosure Agreement":
            params = nda_parameters()
        elif doc_type == "Rental Agreement":
            params = rental_parameters()
        elif doc_type == "Employment Contract":
            params = employment_parameters()
        else:
            params = generic_parameters()

        if st.button("Generate Document", type="primary"):
            with st.spinner("Generating document..."):
                generate_document(doc_type, jurisdiction, params)

    with col2:
        st.header("Preview & Verification")

        if 'generated_doc' in st.session_state:
            display_document(st.session_state.generated_doc)
            display_verification(st.session_state.verification)

def nda_parameters():
    """Input fields for NDA parameters."""
```

```
    params = {}
    params['disclosing_party'] = st.text_input("Disclosing Party Name")
    params['receiving_party'] = st.text_input("Receiving Party Name")
    params['effective_date'] = st.date_input("Effective Date")
    params['duration_years'] = st.number_input("Confidentiality Duration (years)", 1, 10, 3)
    params['purpose'] = st.text_area("Purpose of Disclosure")
    return params

def rental_parameters():
    """Input fields for rental agreement parameters."""
    params = {}
    params['landlord_name'] = st.text_input("Landlord Name")
    params['tenant_name'] = st.text_input("Tenant Name")
    params['property_address'] = st.text_area("Property Address")
    params['monthly_rent'] = st.number_input("Monthly Rent (INR)", 0, 1000000, 25000)
    params['security_deposit'] = st.number_input("Security Deposit (INR)", 0, 5000000, 100000)
    params['lease_term_months'] = st.number_input("Lease Term (months)", 1, 60, 11)
    return params

def employment_parameters():
    """Input fields for employment contract parameters."""
    params = {}
    params['employer_name'] = st.text_input("Employer Name")
    params['employee_name'] = st.text_input("Employee Name")
    params['position'] = st.text_input("Position/Designation")
    params['annual_salary'] = st.number_input("Annual Salary (INR)", 0, 50000000, 600000)
    params['start_date'] = st.date_input("Start Date")
    params['probation_months'] = st.number_input("Probation Period (months)", 0, 12, 3)
    return params

def generic_parameters():
```

```
    """Generic parameters for other document types."""
    params = {}
    params['party_a'] = st.text_input("Party A Name")
    params['party_b'] = st.text_input("Party B Name")
    params['effective_date'] = st.date_input("Effective Date")
    params['description'] = st.text_area("Document Description")
    return params

def generate_document(doc_type, jurisdiction, params):
    """Execute the full document generation pipeline."""
    components = load_components()

    # Step 1: Research
    with st.status("Researching legal provisions...", expanded=True) as status:
        research = components['researcher'].research(
            doc_type, jurisdiction, list(params.values())
        )
        st.write(f"Retrieved {len(research.get('applicable_statutes', []))} statutes")
        status.update(label="Research complete", state="complete")

    # Step 2: Draft
    with st.status("Generating document draft...", expanded=True) as status:
        document = components['drafter'].draft(doc_type, params, research)
        st.write(f"Generated {len(document['sections'])} sections")
        status.update(label="Draft complete", state="complete")

    # Step 3: Verify citations
    with st.status("Verifying citations...", expanded=True) as status:
        verification = components['verifier'].verify_document(document)

        if verification['flagged_count'] > 0:
```

```
            st.warning(f"{verification['flagged_count']} citation(s) require human verification")
        else:
            st.success("All citations verified")

        status.update(label="Verification complete", state="complete")

    # Store in session state
    st.session_state.generated_doc = document
    st.session_state.verification = verification
    st.session_state.research = research

def display_document(document):
    """Display generated document with AI disclosure."""
    st.warning(document['metadata']['ai_disclosure'])

    for section in document['sections']:
        with st.expander(section['name'], expanded=True):
            st.markdown(section['content'])

            if section.get('requires_review'):
                st.error("⚠ This section requires mandatory human review")

def display_verification(verification):
    """Display verification results."""
    st.subheader("Citation Verification")

    if verification['overall_status'] == 'ALL_VERIFIED':
        st.success(f"✅ All {verification['verified_count']} citations verified")
    elif verification['overall_status'] == 'REQUIRES_HUMAN_VERIFICATION':
        st.error(f"⚠ {verification['flagged_count']} citation(s) require human verification")
```

```
        for citation in verification['flagged_citations']:
            with st.expander(f"Flagged: {citation['text'][:50]}..."):
                st.json(citation['verification'])
                st.error("MANDATORY: Verify this citation independently before use")

if __name__ == "__main__":
    main()
```

5.7 Template Engine

```
python
# core/templates.py
import yaml
from jinja2 import Template, Environment, FileSystemLoader
from typing import Dict, List

class TemplateEngine:
    """Manages legal document templates with Jinja2 rendering."""

    def __init__(self, template_dir: str = "templates/"):
        self.env = Environment(
            loader=FileSystemLoader(template_dir),
            trim_blocks=True,
            lstrip_blocks=True
        )
        self.template_dir = template_dir

    def load(self, document_type: str) -> Dict:
        """Load a template definition from YAML."""
        filename = document_type.lower().replace(" ", "_") + ".yaml"
```

```
        filepath = self.template_dir + filename

        with open(filepath, 'r') as f:
            return yaml.safe_load(f)

    def render_section(self, section_def: Dict, variables: Dict) -> str:
        """Render a single section with variables."""
        if 'template_string' in section_def:
            template = Template(section_def['template_string'])
            return template.render(**variables)
        return section_def.get('content', '')
```

Example Template (nda.yaml):

```
yaml
name: "Non-Disclosure Agreement"
jurisdiction: "India"
sections:
  - name: "Parties"
    template_string: |
      This Non-Disclosure Agreement ("Agreement") is entered into as of
      {{ effective_date }} by and between:

      **{{ disclosing_party }}** ("Disclosing Party"), and
      **{{ receiving_party }}** ("Receiving Party").
    ai_enhance: false
    human_review: false

  - name: "Confidential Information"
    template_string: |
```

```
    "Confidential Information" means all non-public information disclosed
    by the Disclosing Party to the Receiving Party, whether orally or in
    writing, that is designated as confidential or that reasonably should
    be understood to be confidential given the nature of the information
    and circumstances of disclosure.

    Purpose: {{ purpose }}
  ai_enhance: true
  human_review: false

- name: "Obligations"
  template_string: |
    The Receiving Party shall:
    1. Hold Confidential Information in strict confidence
    2. Not disclose to third parties without prior written consent
    3. Use Confidential Information solely for the Purpose
    4. Return or destroy all Confidential Information upon request
  ai_enhance: false
  human_review: false

- name: "Term"
  template_string: |
    This Agreement shall remain in effect for a period of
    {{ duration_years }} year(s) from the Effective Date.
  ai_enhance: false
  human_review: false


---
```

6. Regulatory Compliance

6.1 Supreme Court AI Regulations Compliance

The Supreme Court of India's draft Regulations for Use of Artificial Intelligence in Courts, 2026 establish key requirements that LegalEase addresses :

Requirement	LegalEase Implementation
Human oversight	Mandatory review gates before finalization
Accountability	Audit trail logging all generations and approvals
Transparency	AI disclosure on all generated content
Verification	Citation verification agent with verification status
No reliance on unverified AI	Flagged citations require explicit human confirmation

The Supreme Court has held that a decision based on fake or hallucinated material is "no decision at all" . LegalEase's verification architecture directly addresses this by ensuring no citation is presented as verified without independent confirmation.

6.2 Bar Council of India Compliance

The BCI's digital ethics circular requires disclosure when AI tools generate or substantially modify legal content . LegalEase implements:

· AI disclosure statements on all generated documents

· Content origin marking indicating which sections are AI-generated

· No fabricated content — all citations must be verified or flagged

· User undertakings acknowledging responsibility for verification

6.3 Data Protection Compliance

LegalEase complies with the Digital Personal Data Protection Act, 2023 through:

· Data minimization (only collect necessary parameters)

· Local-first architecture option (documents never leave user's infrastructure)

· Encryption at rest and in transit

· No training on user documents without explicit consent